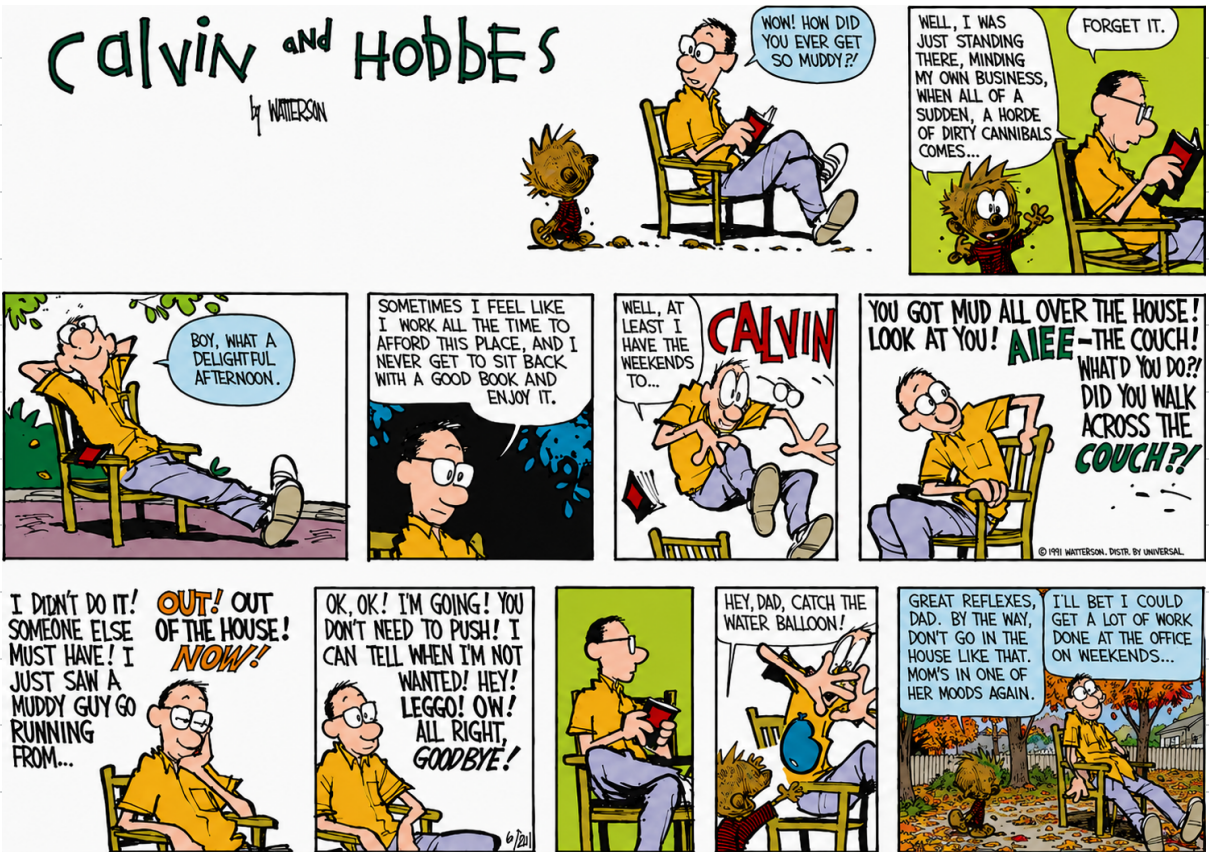
Advanced Agent Patterns: ReAct, Reflection & Memory

Good evening!! We will start 8:10 PM



Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ✗

Chain of Thought Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

LLM: Q: _ _ A: _ _ .

(1) COT: Q:

A: <ST-RES> _ _ _ _ _ _ _ _ </ST-RES>

OUTPUT TOKEN </EOS>

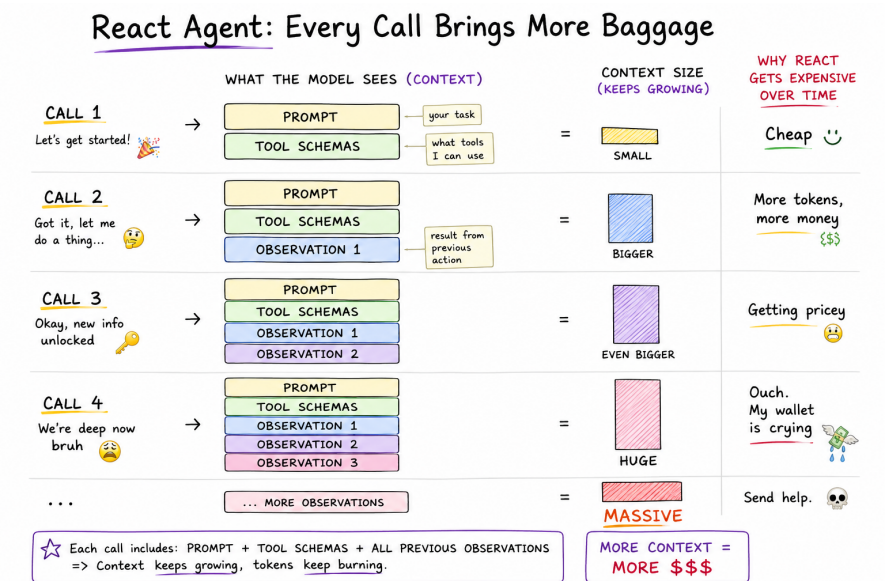
(2) ReAct

Did our store hit the Q2 Target?
Target: Revenue must grow by 20% from Q1 to Q2

Thought: I need q1, q2, and compute percentage.
Action: query-sales (q1)
Observation: q1 = 10k

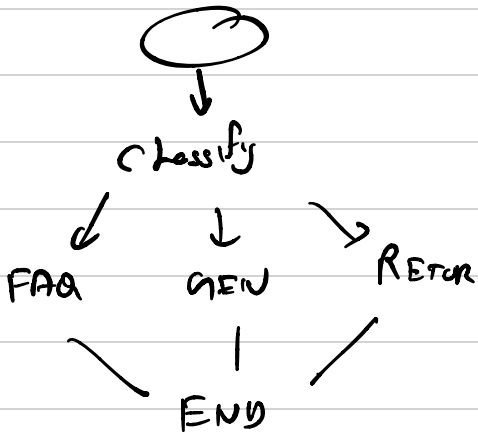
Thought...

Thrashing:
1. thought: I should seach q2 results; action: seach_web(q2 sales target); output: irrelevant result
2. thought: I should search q2 results; action: search_web(store revenue q2 sales); irrelevant result.



3 Plan & Solve

1. get Q1 revenue
2. get Q2 revenue
3. Get growth target
4. Compute difference
5. Compute percentage



Frameworks, Workflows work very well!!
Thumb rules, KISS, keep it sweet and simple.

ReWOO
Reason without Observation.

1. get Q1 revenue -> {"tool_name", "args"}
2. get Q2 revenue -> {"tool_name", "args"}
3. Get growth target -> {"tool_name", "args"}
4. Compute difference -> {"tool_name", "args"}
5. Compute percentage -> {"tool_name", "args"}

4. Self Consistency

You ask same question to a llm with high temperature. Do a majority voting on the answer.

Reflection

1. Self-Refine

includes same LLM reviewing the answer. Tone of answer, temperament (the flow of words)

2. Reflexion

Agent tries a task
Environment gives the feedback
Agent writes a lesson
Lesson is stored in memory
Agent retries with learnt lesson.

Corner - Tests

3. CRITIC

Judging output by TOOL-CALLS,

Error/success result from tool-call can act as review.

What's the current algorithm for linear topology:
web_search (linear topology)

4. LLM as a judge

Mostly useful for subjective/semi-subjective kind of answer.

don't just use LLM with system prompt of "Review following answer"

a. empathy
b. Accuracy
c. tone